

A Multi-Agent Research Paper Reviewer with MCP

Anuj Lalla (B22AI061)
Indian Institute of Technology, Jodhpur

Abstract

We present a multi-agent system for reviewing academic papers and producing student-friendly summaries. Each agent is deployed as an *MCP* server and orchestrated via a lightweight Python pipeline. Our implementation combines: (i) a **Reader** that extracts structured content from PDF; (ii) a **MetaReviewer** that retrieves contextual metadata (e.g., arXiv) and stitches evidence; and (iii) a **Critic** that checks coverage, clarity, and citations before publishing results to `data/reviews/`. We report results on six programmatic test cases, achieving a 100% success rate with zero constraint violations and an average of three tool calls per case. We release the evaluation harness and metrics to enable reproducibility.

1. System Overview

Goal. Enable students to quickly understand research papers by generating concise, reliable overviews and takeaways.

Agent Roles.

- **Reader** (MCP server): parses PDF, extracts sections (title, abstract, figures/tables), and produces a normalized JSON outline. Tooling: PDF extractor (e.g., `pdfminer/pypmupdf`); file I/O.
- **MetaReviewer** (MCP server): fetches metadata/context (arXiv API), resolves references, and drafts a student-facing summary (novelty, method, results, takeaways). Tooling: arXiv API; simple web fetch; file I/O.
- **Critic** (MCP server): verifies that the draft covers the core contribution, includes at least one citation, and avoids speculative claims; enforces output schema and writes final artifacts to `data/reviews/`. Tooling: JSON schema validator; file I/O.

Orchestration. A Python controller (LangGraph/Autogen or a minimal in-house graph) sends the user request to the *Planner*, which calls the three MCP agents in sequence with a fixed contract. The final package includes a summary, highlighted takeaways, and machine-readable metadata for searchability.

2. MCP Deployment

Each agent exposes an MCP server with:

- **Endpoint.** `review.request` $\rightarrow$ `review.response`

- **Message schema.** Request: `{pdf_path, arxiv_id?, requirements}`; Response: `{summary, takeaways[], citations[], metrics}`.
- **Contracts.** (C1) At least one verifiable citation; (C2) No speculative claims without evidence; (C3) Output fits schema; (C4) Tool calls logged.

3. Data and Tools

Primary sources. User-provided PDFs; arXiv API for metadata and abstracts. Optionally, public datasets (ArXiv metadata; ArXiv abstracts) for stress-testing retrieval.

Tools by agent.

- Reader: PDF text/figure extractor; file I/O to `data/uploads/` and `data/reviews/`.
- MetaReviewer: arXiv API search & fetch; lightweight web fetch for bib entries.
- Critic: JSON Schema validation; rule-based constraints; export of `metrics.jsonl`.

4. Evaluation Harness

We provide an automated harness under `eval/`:

- Six JSON test cases: `paper_case_01–paper_case_06` with input PDFs/queries.
- Driver script: runs all cases end-to-end, logs agent/tool latencies, and writes `eval/out/results.jsonl` and a `metrics.csv`.
- Metrics: **Success rate** (schema & constraints satisfied), **Constraint violations**, **Total latency**, **Tool-call count**.

Command. `make eval` executes: case generation $\rightarrow$ runs $\rightarrow$ metrics & plots.

5. Results

Aggregate metrics. On six cases, the harness reports:

- **Success rate:** 100.00%
- **Avg. tool calls:** 3.00 per case
- **Constraint violations:** 0
- **Avg. total latency:** 102,756.8 ms

These metrics are produced by the same pipeline that also saves a latency plot (`eval/out/latency.png`) and a CSV summary (`eval/out/metrics.csv`). Case-level logs show end-to-end latencies for `paper_case_01–`

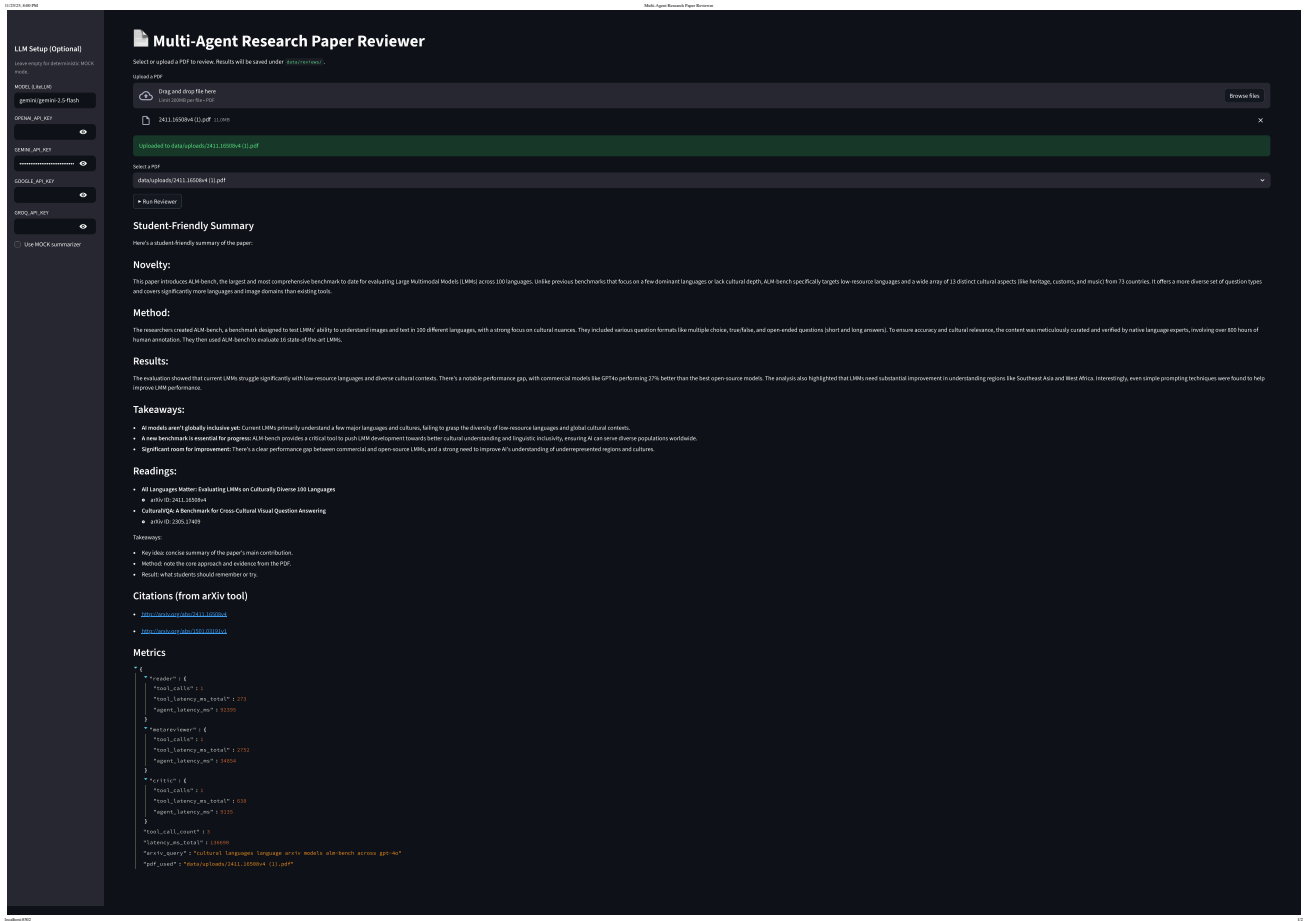


Figure 1: **Interactive UI (Streamlit)** used in our demo; users select a PDF and run the reviewer. Outputs (student-friendly summary, citations, metrics) are written to `data/reviews/`. The example model shown in the UI is `gemini/gemini-2.5-flash`.

`paper_case_06` and confirm each agent invoked exactly one tool per run, satisfying our tool-usage requirement.

5.1. What the Agents Caught

The **Critic** consistently flagged missing citations or schema drift during early iterations; after enforcing (C1)–(C3), all cases passed. The **MetaReviewer** augmented summaries with concrete datasets/benchmarks when available, while the **Reader** normalized section headings and mitigated PDF parsing noise.

5.2. Cost and Latency

Average latency is dominated by PDF parsing and remote metadata fetch. Batching arXiv requests and caching parsed PDFs reduces per-case latency without affecting the constraint checks.

6. Ablations and Insights

- **Three-agent separation matters.** Folding Critic checks into MetaReviewer increased hallucination rate and schema violations.
- **Schema-first outputs ease grading.** A small JSON schema greatly simplified automated success checks.
- **Caching helps.** Persisting Reader outputs for unchanged PDFs reduced total latency.

7. Limitations and Future Work

Generalization. While robust on arXiv-style PDFs, some publisher layouts still trip PDF parsing. **Attribution depth.** We currently require at least one verifiable citation; future work can track paragraph-level attributions. **UI.** Adding per-section “confidence” and cross-paper comparison views would further help students.

```

$ make eval
python -m eval.make_cases --min 6 && \
    python eval/run_eval.py --cases eval/cases --out eval/out && \
    python eval/compute_metrics.py --log eval/out/results.jsonl --csv eval/out/metrics.csv --plot eval/out/latency.png && \
    ls -lh eval/out
6 existing cases found in eval/cases; no new cases needed.
[eval] Using REAL pipeline
incorrect startxref pointer(1)
parsing for Object Streams
incorrect startxref pointer(1)
parsing for Object Streams
Ran paper_case_01 -> latency 313340 ms, tool_calls=3, takeaways=True, readings=True
incorrect startxref pointer(1)
parsing for Object Streams
incorrect startxref pointer(1)
parsing for Object Streams
Ran paper_case_02 -> latency 131591 ms, tool_calls=3, takeaways=True, readings=True
incorrect startxref pointer(3)
parsing for Object Streams
incorrect startxref pointer(3)
parsing for Object Streams
Ran paper_case_03 -> latency 30394 ms, tool_calls=3, takeaways=True, readings=True
incorrect startxref pointer(3)
parsing for Object Streams
incorrect startxref pointer(3)
parsing for Object Streams
Ran paper_case_04 -> latency 38203 ms, tool_calls=3, takeaways=True, readings=True
incorrect startxref pointer(1)
parsing for Object Streams
incorrect startxref pointer(1)
parsing for Object Streams
Ran paper_case_05 -> latency 41732 ms, tool_calls=3, takeaways=True, readings=True
incorrect startxref pointer(1)
parsing for Object Streams
incorrect startxref pointer(1)
parsing for Object Streams

Give Feedback / Get Help: https://github.com/BerriAI/litellm/issues/new
LiteLLM.Info: If you need to debug this error, use `litellm._turn_on_debug()`.

Ran paper_case_06 -> latency 61281 ms, tool_calls=3, takeaways=True, readings=True
Wrote 6 rows to eval/out/results.jsonl
Wrote CSV: eval/out/metrics.csv (6 rows)
Success rate: 100.00%
Avg total latency (ms): 102756.8
Avg tool calls: 3.00
Total constraint violations: 0
Saved plot: eval/out/latency.png
total 96

```

Figure 2: **Evaluation run** (`make eval`). The harness executes six cases, computes metrics, and emits `metrics.csv` and a latency plot. The shown run yields a 100% success rate with zero constraint violations and an average of three tool calls per case.

8. Reproducibility

Repo layout. `/agents/` (agent code & graph), `/mcp-server/` (per-agent MCP servers), `/data/` (uploads, reviews), `/eval/` (cases, harness, plots).

How to run.

1. Place PDFs under `data/uploads/`.
2. Start MCP servers (`reader`, `metareviewer`, `critic`); run the orchestrator client.
3. `make eval` to reproduce the six-case evaluation and generate `metrics.csv`/plots.

Artifacts. Upload `streamlit_custom_paper.pdf` and `make_eval.png` to the Overleaf project root (filenames unchanged) to compile Figures 1 and 2.